Adevar Labs

Prepared for:
M0

January 21, 2026

M0 EVM & Solana Portal Audit Report

Version: after-fix report

Table of Contents

1. Executive Summary **3**

About M0 Portal 3

Audit Summary 3

Overall Security Posture 3

Launch Recommendations 4

Audit Scope 4

2. Assumptions and Considerations **5**

Audit Assumptions 5

Centralization Risks 5

Trust Assumptions 5

3. Severity Definitions **6**

Impact 6

Likelihood 6

Severity Classification Matrix 6

4. Findings **8**

L01: Insufficient balance for first depositor on Hub when rounding error is covered 8

L02: Missing whenNotPaused modifier allows sendEarnersMerkleRoot to operate during protocol pause 10

L03: Missing hubChainId_ constructor parameter will not allow Spoke Portal upgrades 12

L04: Missing crossSpokeTransferEnabled will not allow Spoke Portal upgrades 13

5. Enhancement Opportunities **14**

E01: Ensuring consistent event emissions will improve visibility 14

About Us **15**

About Adevar Labs 15

Audit Methodology 15

Confidentiality Notice 16

Legal Disclaimer 16

1. Executive Summary

About M0 Portal

Portal is a cross-chain component that extends M0's stablecoin infrastructure across multiple blockchains. M0's primary goal is to enable projects to easily deploy their own stablecoins, and Portal ensures these assets can move seamlessly between chains.

The system is architected around three main components: a HubPortal deployed on Ethereum, which acts as the source of truth and manages token locking/releasing. SpokePortals on other chains that mint and burn tokens based on messages received from the Hub. BridgeAdapters that provide an abstraction layer allowing the protocol to work with different bridging providers.

Two repositories were in scope during the audit:

- m-portal-v2: Solidity implementation for EVM chains (Hub/Spoke)
- solana-portal: Solana implementation (Spoke)

Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	–	–
High	0	–	–
Medium	0	–	–
Low	4	3	1

Enhancement opportunities: 1

Overall Security Posture

This section describes the security posture before and after the fix review. The fix review is the stage of the audit where Adevar Labs verified the fixes implemented by the M0 development team for the issues identified during the audit.

After Fix Review

The fix review confirmed the resolution of all issues: 3 have been fixed, and 1 has been acknowledged with no risk identified after further investigation.

Before Fix Review Summary

The audit of the M0 Portal repositories revealed no critical or high-severity issues, highlighting the team's strong development practices and security awareness.

A total of 4 low severity issues and 1 enhancement opportunity were identified during this audit.

Two issues involved missing parameters in the SpokePortal upgrade function (crossSpokeTransferEnabled and hubChainId_), preventing successful upgrades. The sendEarnersMerkleRoot function was missing the whenNotPaused modifier, creating inconsistency with other protocol functions. A balance mismatch issue could cause DoS for first depositors when rounding errors are covered from yield. Additionally, inconsistent event emissions were noted between EVM and Solana implementations.

Launch Recommendations

To ensure a smooth and secure launch, we offer the following recommendations:

I. Recommended:

- Resolve the low severity issues and enhancement opportunities.
- Thoroughly simulate and test the deployment procedure to ensure smooth migration.

II. Post-Launch Monitoring:

- Track cross-chain message delivery rates and latency between Hub and all Spoke chains, with alerts for anomalies.
- Monitor bridge balance discrepancies and state synchronization issues across chains.
- Establish dashboards tracking bridged volumes, user activity patterns, and protocol health metrics for both EVM and Solana deployments.

Audit Scope

Repositories:

m-portal-v2

- https://github.com/m0-foundation/m-portal-v2
- **Before Fix Review Commit Hash:** PR#24 at d9bc5194d2ce78d50a80d411f6143e6034bcca81
- **After Fix Review Commit Hash:** 04514515d44bdf051cd5f996ef8cda7a0ac64187
- **Files/Modules in Scope:**
  - *evm/src*
  - *evm/script*
  - *evm/test/fork/migrate*

solana-portal

- https://github.com/m0-foundation/solana-portal
- **Before Fix Review Commit Hash:** PR#17 at a0e73c74a7705f534389c765f0955f2173ba64a0
- **After Fix Review Commit Hash:** a0e73c74a7705f534389c765f0955f2173ba64a0 (no issue found)
- **Files/Modules in Scope:**
  - *programs/hyperlane-adapter/src*
  - *programs/portal/src*
  - *programs/wormhole-adapter/src*

2. Assumptions and Considerations

Audit Assumptions

- The M Token implementation, including its earning index calculation, accrual mechanism, and mint/burn authorization logic, is assumed to function correctly.
- Similarly, the token wrapping and unwrapping mechanisms that convert between M tokens and extension tokens operate correctly, including multiplier calculations for interest-bearing conversions.
- The project is expected to set the proper consistency level parameters during Wormhole Adapter initialization.

Centralization Risks

- The DEFAULT_ADMIN_ROLE (EVM) on Portal and Bridge Adapter contracts can authorize UUPS upgrades without timelock, multisig, or governance approval.
- The OPERATOR_ROLE (EVM) controls numerous protocol parameters including default bridge adapter selection, supported bridging paths, gas limits, and cross-spoke transfer enablement.
- PAUSER_ROLE (EVM) and admin accounts (Solana) can halt all outbound bridging operations indefinitely.
- Each Solana program (Portal, Wormhole Adapter, Hyperlane Adapter) has a single admin account with full authority over pause state, peer configuration, and ISM/IGP and LUT settings.

Trust Assumptions

- The underlying bridge protocols (Wormhole and Hyperlane) are assumed to be secure and trusted.
- The protocol relies on external validator sets for message authenticity:
  - Wormhole: Guardian network
  - Hyperlane: Validator set and Interchain Security Module (ISM) configuration

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements–typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue–particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

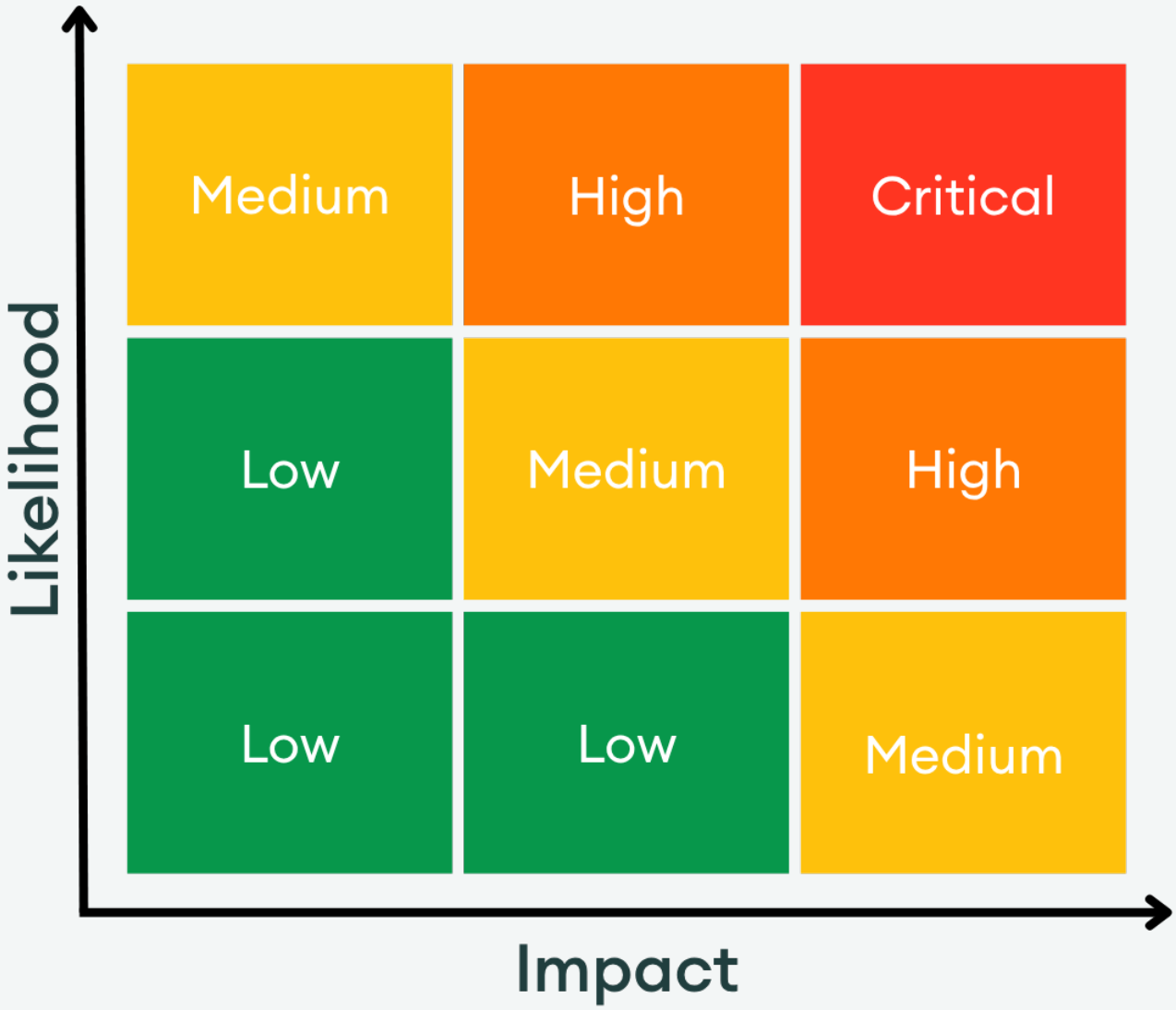
- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substansial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability
- **Low**: Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

L01: Insufficient balance for first depositor on Hub when rounding error is covered

Status: Acknowledged

Impact: Low Medium High

Likelihood: Low Medium High

Severity: Low

Location: https://github.com/m0-foundation/m-portal-v2/blob/d9bc5194d2ce78d50a80d411f6143e6034bcca81/evm/src/Portal.sol#L581-L596

>_ Portal.sol SOLIDITY

```
579:         uint256 actualAmount = _mBalanceOf(address(this)) - startingBalance;
580:
581:         if (specifiedAmount > actualAmount) {
582:             unchecked {
583:                 // If the difference between the specified transfer amount and the act
        ual amount exceeds
584:                 // the maximum acceptable rounding error (e.g., due to fee-on-transfer
         in an extension token)
585:                 // transfer the actual amount, not the specified.
586:
587:                 // Otherwise, the specified amount will be transferred and the deficit
         caused by rounding error will
588:                 // be covered from the yield earned by HubPortal.
589:                 if (specifiedAmount - actualAmount > _getMaxRoundingError()) {
590:                     // Ensure that updated transfer amount is greater than 0
591:                     _revertIfZeroAmount(actualAmount);
592:                     return actualAmount;
593:                 }
594:             }
595:         }
596:         return specifiedAmount;
597:     }
598:
```

Description:

When a user swaps principal to M token and the rounding error is within the acceptable threshold _getMaxRoundingError(), the protocol covers the deficit from yield.
However, this creates a mismatch between the actual balance and the amount sent to the spoke chain.

1. User swaps 100e18 principal → M token
2. Actual received: 100e18 - 2 wei (due to rounding/fee-on-transfer)
3. _getTransferAmount() returns 100e18 (since 2 wei <= _getMaxRoundingError())
4. Hub tracks bridgedPrincipal based on 100e18

5. Hub's actual M token balance: 100e18 - 2 wei
6. Spoke mints full 100e18 to user
7. User sends back 100e18 from spoke
8. Hub tries to transfer 100e18 but only has 100e18 - 2 wei → transfer fails

This results in a DoS for the first depositor when a rounding error occurs.

Recommendation:

If covering rounding errors from yield is intentional, pre-fund the Portal contract with a small buffer to cover rounding errors for early depositors before yield accrues.

This prevents DoS for first depositors and ensures transfers don't fail due to insufficient balance.

Developer Response:

Acknowledged. The HubPortal is already deployed will not have a zero balance. If re-deployed, we will seed it with a small amount of tokens to avoid this.

L02: Missing whenNotPaused modifier allows sendEarnersMerkleRoot to operate during protocol pause

Status: Resolved

Impact: Low Medium High

Likelihood: Low Medium High

Severity: Low

Location: https://github.com/m0-foundation/m-portal-v2/blob/d9bc5194d2ce78d50a80d411f6143e6034bcca81/evm/src/HubPortal.sol#L162-L183

>_ HubPortal.sol SOLIDITY

```
160:
161:     /// @inheritdoc IHubPortal
162:     function sendEarnersMerkleRoot(
163:         uint32 destinationChainId,
164:         bytes32 refundAddress,
165:         bytes calldata bridgeAdapterArgs
166:     ) external payable returns (bytes32 messageId) {
167:         address bridgeAdapter = defaultBridgeAdapter(destinationChainId);
168:         _revertIfZeroBridgeAdapter(destinationChainId, bridgeAdapter);
169:
170:         return _sendEarnersMerkleRoot(destinationChainId, refundAddress, bridgeAdapter
        , bridgeAdapterArgs);
171:     }
172:
173:     /// @inheritdoc IHubPortal
174:     function sendEarnersMerkleRoot(
175:         uint32 destinationChainId,
176:         bytes32 refundAddress,
177:         address bridgeAdapter,
178:         bytes calldata bridgeAdapterArgs
179:     ) external payable returns (bytes32 messageId) {
180:         _revertIfUnsupportedBridgeAdapter(destinationChainId, bridgeAdapter);
181:
182:         return _sendEarnersMerkleRoot(destinationChainId, refundAddress, bridgeAdapter
        , bridgeAdapterArgs);
183:     }
184:
185:     /// @inheritdoc IHubPortal
```

Description:

The sendEarnersMerkleRoot function lacks the whenNotPaused modifier, making it the only message-sending function that can execute while the protocol is paused.
All other send functions consistently enforce pause state.

This creates an inconsistency where earner Merkle root updates can be sent cross-chain even during emergency pauses.

Notably, the Solana portal implementation correctly includes pause state checks in its

SendMerkleRoot instruction, creating a discrepancy between the two implementations.

Recommendation:

Add the whenNotPaused modifier to both sendEarnersMerkleRoot functions for consistency.

Developer Response:

Fixed according to the recommendation.
The sendEarnersMerkleRoot functions now have the pause modifier.

L03: Missing hubChainId_ constructor parameter will not allow Spoke Portal upgrades

Status: Resolved

Impact: Low Medium High

Likelihood: Low Medium High

Severity: Low

Location: https://github.com/m0-foundation/m-portal-v2/blob/d9bc5194d2ce78d50a80d411f6143e6034bcca81/evm/script/migrate/MigrateSpokePortalBase.sol#L30

>_ MigrateSpokePortalBase.sol SOLIDITY

```
28:     function _upgradeToPortalV2() internal virtual {
29:         bytes memory initializeData = abi.encodeCall(SpokePortal.initialize, (ADMIN_V2
        , PAUSER_V2, OPERATOR_V2));
30:         address spokePortalV2Implementation = address(new SpokePortal(M_TOKEN, REGISTR
        AR, SWAP_FACILITY, ORDER_BOOK));
31:         UUPSUpgradeable(PORTAL).upgradeToAndCall(spokePortalV2Implementation, initiali
        zeData);
32:     }
```

Description:

The SpokePortal V2 implementation constructor uses a new uint32 parameter named hubChainId_ that corresponds to the chain ID of the Hub portal. However, the _upgradeToPortalV2 function, which upgrades SpokePortals to the Portal V2 implementation, does not include this parameter in the constructor arguments of the SpokePortal.

Recommendation:

Include the hubChainId_ in the constructor arguments of the SpokePortal.

Developer Response:

Fixed according to the recommendation.

L04: Missing crossSpokeTransferEnabled will not allow Spoke Portal upgrades

Status: Resolved

Impact: Low Medium High

Likelihood: Low Medium High

Severity: Low

Location: https://github.com/m0-foundation/m-portal-v2/blob/d9bc5194d2ce78d50a80d411f6143e6034bcca81/evm/script/migrate/MigrateSpokePortalBase.sol#L29

>_ MigrateSpokePortalBase.sol SOLIDITY

```
27:     /// @dev Upgrade SpokePortal to Portal V2 implementation AFTER storage has been cl
        eared
28:     function _upgradeToPortalV2() internal virtual {
29:         bytes memory initializeData = abi.encodeCall(SpokePortal.initialize, (ADMIN_V2
        , PAUSER_V2, OPERATOR_V2));
30:         address spokePortalV2Implementation = address(new SpokePortal(M_TOKEN, REGISTR
        AR, SWAP_FACILITY, ORDER_BOOK));
31:         UUPSUpgradeable(PORTAL).upgradeToAndCall(spokePortalV2Implementation, initiali
        zeData);
```

Description:

The SpokePortal V2 implementation introduces a new boolean parameter named crossSpokeTransferEnabled, which enables portals deployed on spokes to transfer tokens amongst themselves without routing through the Hub portal. However, the _upgradeToPortalV2 function, which upgrades SpokePortals to the Portal V2 implementation, does not include this parameter in the call to SpokePortal.initialize.

Recommendation:

Include the crossSpokeTransferEnabled parameter in the SpokePortal.initialize call.

Developer Response:

Fixed according to the recommendation.

5. Enhancement Opportunities

E01: Ensuring consistent event emissions will improve visibility

Location: https://github.com/m0-foundation/solana-portal/blob/a0e73c74a7705f534389c765f0955f2173ba64a0/programs/portal/src/instructions/enable_cross_spoke_transfers.rs#L19

>_ enable_cross_spoke_transfers.rs RUST

```rust
17: }
18:
19: impl EnableCrossSpokeTransfers<'_> {
20:     pub fn handler(ctx: Context<Self>) -> Result<()> {
21:         ctx.accounts.portal_global.isolated_hub_chain_id = None;
```

Description:

The v2 portals in both EVM and SVM are adding a new function for enabling cross-spoke token transfers. There is a minor inconsistency regarding the event emissions in the two implementations. On the EVM side there is an event emission, while on the SVM there is no event emission.

EVM

SOLIDITY

```solidity
function enableCrossSpokeTokenTransfer() external onlyRole(OPERATOR_ROLE) {
      if (_getSpokePortalStorageLocation().crossSpokeTokenTransferEnabled) return;

      _getSpokePortalStorageLocation().crossSpokeTokenTransferEnabled = true;
      emit CrossSpokeTokenTransferEnabled(); //Event emitted here
  }
```

SVM

RUST

```rust
impl EnableCrossSpokeTransfers<'_> {
    pub fn handler(ctx: Context<Self>) -> Result<()> {
        ctx.accounts.portal_global.isolated_hub_chain_id = None;
        Ok(())
    } //No event emission
}
```

Potential Benefit:

Improved protocol visibility and potential for external integrations relying on events.

Recommendation:

Emit an event in the SVM implementation to reflect the state change of cross-spoke transfers on the spoke chain.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)

- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in

connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.